Python Syllabus

Phase 1: The Absolute Basics & Control Flow

The Core Concepts:

- **Environment Setup:** Configuring VS Code, working with the Python interactive shell, and running your first script.
- **Variables & Core Types:** Working with integers, floats, strings, and booleans. Master dynamic typing and explicit type casting (e.g., int("42")).
- **Basic Arithmetic:** Operators for calculations (+, -, *, /, //, %,).
- **Conditional Logic:** Controlling execution flow using if, elif, and else blocks with logical operators (and, or, not).
- **Loops & Iteration:** Repeating code blocks using while and for loops. Managing loops with break, continue, and pass.

Project: The Smart ATM Simulator

- **The Goal:** Build a CLI tool that mimics a physical ATM.
- **What you will code:** Initialize a user balance and a hardcoded 4-digit security PIN. Use a while loop to grant a maximum of 3 login attempts. Once authenticated, use an infinite loop and conditionals to process selections for checking balance, depositing money, and validating withdrawals so the account never drops below zero.

Phase 2: Data Structures & Collections

The Core Concepts:

- **Lists:** Working with mutable ordered sequences. Master index slicing, core methods (append(), pop(), sort()), and concise list comprehensions.
- **Tuples:** Fixed, immutable sequences. Understanding tuple packing, unpacking, and when to use them for data integrity.
- **Dictionaries:** Key-value mapping, hashing mechanics, fast lookups, and iterating through .keys(), .values(), and .items().
- **Sets:** Unordered collections of unique items. Performing set algebra like unions, intersections, and differences.

Project: The Contact Book Manager

- **The Goal:** Create an in-memory, searchable contact database.
- **What you will code:** Store your directory inside a dictionary where the keys are unique contact names, and the values are nested dictionaries containing custom data fields: {'phone': str, 'email': str}. Write code to handle adding new profiles, searching case-insensitively, modifying details, and listing all entries cleanly sorted in alphabetical order.

Phase 3: Functions & Modular Design

The Core Concepts:

- **Functional Anatomy:** Creating reusable code blocks with def, defining inputs (parameters), and extracting outputs (return).
- **Advanced Arguments:** Mixing positional and keyword arguments, assigning default parameters, and accepting variable inputs using *args and kwargs.
- **Variable Scope:** Navigating the boundaries between global and local scopes, and understanding the global and nonlocal keywords.
- **Lambda Expressions:** Writing anonymous, single-expression functions, especially alongside functional utilities like map(), filter(), and sorted().

Project: E-Commerce Cart Engine

- **The Goal:** Write a modular backend calculation engine for an online store checkout.
- **What you will code:** Construct a primary module calculate_total(cart_items, discount_coupon=None) to process itemized lists. Use kwargs to pass optional, variable operational fees (e.g., shipping_fee=5.00, vat_tax=0.13). Incorporate a inline lambda function inside a built-in filter() to automatically scale down prices or apply flat rate discounts to specific premium items.

Phase 4: File Handling & Robust Error Management

The Core Concepts:

- **File Input/Output:** Safely reading, writing, and appending data to physical text files using the self-closing with open(...) context manager.
- **Exception Handling:** Preventing runtime application crashes. Wrapping risky code blocks inside try, except, else, and finally mechanisms.
- **Custom Errors:** Creating and intentionally triggering errors manually using the raise keyword.

Project: Automated Server Log File Analyzer

- **The Goal:** Parse a raw server log text file, extract critical errors, and produce an automated analytics report.
- **What you will code:** Write a script that opens an external log file line-by-line. Safeguard the operations inside a try-except block to gracefully capture issues if the file is missing (FileNotFoundError). Scan each string for specific tags ([INFO], [WARNING], [ERROR]), aggregate the total counts, and write a neatly formatted breakdown report straight into a new file called log_summary.txt.

Phase 5: Object-Oriented Programming (OOP)

The Core Concepts:

- **Classes & Instances:** Moving from procedural code to object blueprints. Working with the constructor (__init__) and understanding the target instance pointer (self).
- **Attributes & State:** Differentiating between instance-level properties and shared class-level properties.
- **The Four OOP Pillars:**

1. **Encapsulation:** Securing state data by making fields private using double underscores (__private_var) and creating controlled access gateways using getters, setters, and the @property decorator.
2. **Inheritance:** Passing attributes and methods down from parent classes to child classes, and utilizing super() to invoke parent constructors safely.
3. **Polymorphism:** Writing flexible methods that share an identical name but change execution styles across different classes (method overriding and duck typing).
4. **Abstraction:** Constructing strict structure blueprints using the abc module and Abstract Base Classes to enforce layout rules on subclasses.

- **Magic Methods:** Overriding internal dunder behaviors to customize string formatting (__str__) or object equality logic (__eq__).

Project: Text-Based RPG Arena

- **The Goal:** Build an interactive, turn-based battle game featuring deep object structures.
- **What you will code:**
  - Design a core abstract Character base class that completely encapsulates a private __health attribute via strict property getters and setters.
  - Derive distinct Hero and Monster classes using inheritance.
  - Leverage polymorphism by creating specialized subclasses (like a Mage hero or a Dragon monster) that override a standard .attack() method to execute unique behaviors—like casting spells or breathing fire.
  - Tie everything together inside a running loop that instantiates characters and automates turn-based combat rounds until a combatant's health drops to zero.

Final Project : **The Inventory & Invoice Management System (IMS)**

Build a terminal-based system for a retail store. The system tracks products in stock, handles customer checkouts, generates transactional invoices, saves sales history to physical files, and utilizes full object-oriented inheritance and encapsulation to manage its data.